

Lab 1 - Global States and Distributed Snapshots

Ruben Mbiyavanga Nassir Naibi Louis Poutrain
Gregory Dubouloy

29 janvier 2026

Table des matières

1	Introduction	2
2	Architecture et Implémentation	2
2.1	Composants	2
2.2	Algorithme	2
3	Exemple Consistant	3
3.1	Description	3
3.2	Scénario	3
4	Interfaces et Résultats	4
4.1	Interface Formulaire (ui.py)	4
4.2	Interface Graphique (ui_graphical.py)	4
4.3	État Global Capturé	4
4.4	Exemple de Résultat	4
5	Conclusion	4

1 Introduction

Ce rapport présente l'implémentation d'un simulateur d'exécution distribuée basé sur l'algorithme *Chandy-Lamport* pour capturer des snapshots globaux cohérents. Le projet inclut deux interfaces utilisateur (formulaire et graphique) permettant de concevoir et d'exécuter des simulations, ainsi qu'une visualisation espace-temps pour l'analyse.

2 Architecture et Implémentation

2.1 Composants

Le projet s'organise autour de quatre modules : **Simulator** (orchestration, file d'événements), **Process** (état local, implémentation Chandy-Lamport), **Message** (type APP/MARKER), **Event** (tuple $(timestamp, action)$).

2.2 Algorithme

1. L'initiateur enregistre son état et marque tous les canaux entrants.
2. Des marqueurs sont envoyés à tous les voisins.
3. À réception du premier marqueur : enregistrement de l'état et des autres canaux.
4. Messages reçus avant le marqueur sont capturés dans l'état du canal.

3 Exemple Consistant

3.1 Description

Notre exemple met en avant une exécution distribuée avec la propriété suivante : dans la coupure réalisée, **toute conséquence détectable a une causalité causale capturable**.

3.2 Scénario

Considérons 3 processus P_1, P_2, P_3 avec la topologie mesh (tous connectés). Le scénario suit ces étapes :

1. $t = 1$: P_3 envoie un message à P_2 (Msg 3→2).
2. $t = 2$: P_1 envoie un message à P_2 (Msg 1→2).
3. $t = 6$: P_1 initie le snapshot global.
4. $t = 8$: P_2 envoie un message à P_1 (Msg 2→1, message critique).
5. $t = 15$: P_1 envoie un message à P_4 (Msg 1→4).
6. $t = 20$: P_1 envoie un message à P_3 (Msg 1→3).

Le message critique $P_2 \rightarrow P_1$ au temps $t = 8$ est reçu après le démarrage du snapshot en P_1 ($t=6$) et avant la réception du marqueur de retour. Ce message est donc correctement capturé par le snapshot.

La coupure est consistante car tout message ayant une conséquence observable est enregistré en tant que message en transit.

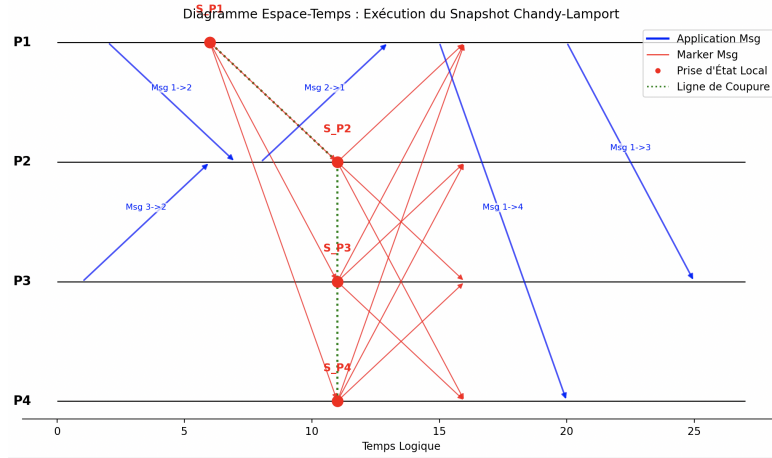


FIGURE 1 – Diagramme espace-temps : snapshot Chandy-Lamport avec message critique.

4 Interfaces et Résultats

4.1 Interface Formulaire (ui.py)

Champs texte pour processus, tableaux pour ajouter/supprimer messages et snapshots. Idéale pour tests rapides.

4.2 Interface Graphique (ui_graphical.py)

Diagramme espace-temps interactif : insertion de messages (clic source → clic destination), snapshots (clic processus), visualisation immédiate.

4.3 État Global Capturé

- Pour chaque processus, on enregistre :
 - Son état local au moment du snapshot.
 - L'état de chaque canal entrant (messages en transit).

4.4 Exemple de Résultat

```
Processus P1:  
  État Local: 3  
  Canaux entrants :  
    From P2: ['Msg 2->1']  
    From P3: <Vide>
```

```
Processus P2:  
  État Local: 5  
  Canaux entrants :  
    From P1: <Vide>  
    From P3: ['Msg 3->2']
```

Ce résultat valide la consistance de la coupure : le message critique de P_2 vers P_1 est bien capturé.

5 Conclusion

Ce projet implémente avec succès l'algorithme *Chandy-Lamport* pour capturer des snapshots globaux cohérents. Les deux interfaces (formulaire et graphique) offrent une grande flexibilité d'utilisation. L'approche garantit la consistance des coupures dans un système asynchrone sans horloge globale.